

服务 n 时, 可以执行这条指令。执行 `syscall` 指令会导致一个到异常处理程序的陷阱, 这个处理程序解析参数, 并调用适当的内核程序。图 8-6 概述了一个系统调用的处理。

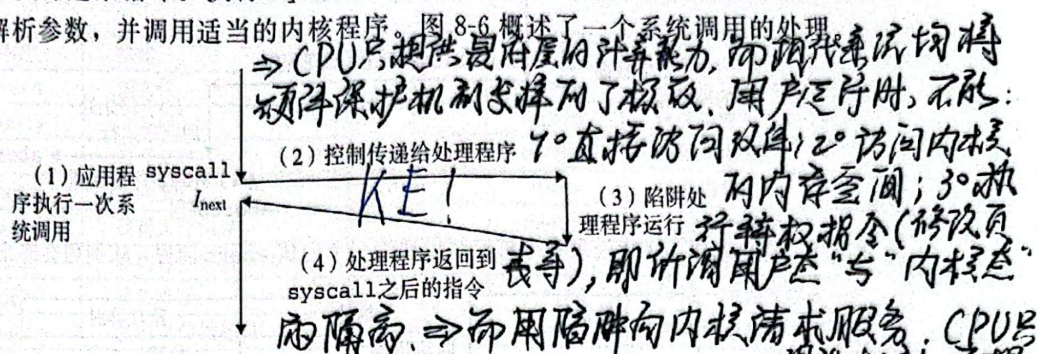


图 8-6 陷阱处理。陷阱处理程序将控制返回给应用程序控制流中的下一条指令。从程序员的角度来看, 系统调用和普通的函数调用是一样的。然而, 它们的实现非常不同。普通的函数运行在用户模式中, 用户模式限制了函数可以执行的指令的类型, 而且它们只能访问与调用函数相同的栈。系统调用运行在内核模式中, 内核模式允许系统调用执行特权指令, 并访问定义在内核中的栈。8.2.4 节会更详细地讨论用户模式和内核模式。

3. 故障

故障由错误情况引起, 它可能能够被故障处理程序修正。当故障发生时, 处理器将控制转移给故障处理程序。如果处理程序能够修正这个错误情况, 它就将控制返回到引起故障的指令, 从而重新执行它。否则, 处理程序返回到内核中的 `abort` 例程, `abort` 例程会终止引起故障的应用程序。图 8-7 概述了一个故障的处理。

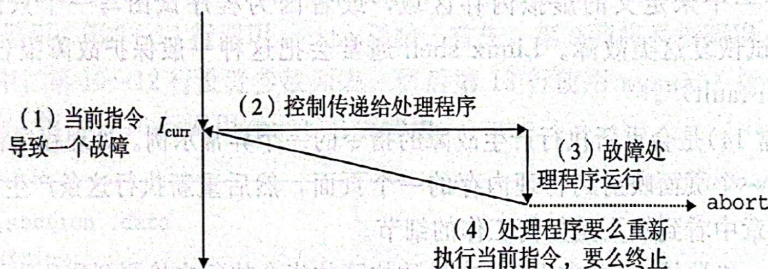


图 8-7 故障处理。根据故障是否能够被修复, 故障处理程序要么重新执行引起故障的指令, 要么终止

一个经典的故障示例是缺页异常, 当指令引用一个虚拟地址, 而与该地址相对应的物理页面不在内存中, 因此必须从磁盘中取出时, 就会发生故障。就像我们将在第 9 章中看到的那样, 一个页面就是虚拟内存的一个连续的块(典型的是 4KB)。缺页处理程序从磁盘加载适当的页面, 然后将控制返回给引起故障的指令。当指令再次执行时, 相应的物理页面已经驻留在内存中了, 指令就可以没有故障地运行完成了。

4. 终止

终止是不可恢复的致命错误造成的结果, 通常是一些硬件错误, 比如 DRAM 或者 SRAM 位被损坏时发生的奇偶错误。终止处理程序从不将控制返回给应用程序。如图 8-8 所示, 处理程序将控制返回给一个 `abort` 例程, 该例程会终止这个应用程序。

8.1.3 Linux/x86-64 系统中的异常

为了使描述更具体, 让我们来看看为 x86-64 系统定义的一些异常。有高达 256 种不同的异常类型[50]。0~31 的号码对应的是由 Intel 架构师定义的异常, 因此对任何 x86-64 系统都是一样的。32~255 的号码对应的是操作系统定义的中断和陷阱。图 8-9 展示了一些示例。

